



Fetch API: Requisições HTTP Modernas em JavaScript

A **Fetch API** é a forma moderna de realizar **requisições HTTP** no JavaScript. Ela substitui o antigo XMLHttpRequest, oferecendo **sintaxe mais simples e baseada em Promises**, tornando o código **mais legível e fácil de manter**.

Promises: Conceito Base

A Fetch API retorna uma **Promise**, que representa uma operação que **pode ser concluída agora ou no futuro**. Isso permite lidar com resultados **assíncronos** de forma clara:

```
fetch(url).then(resposta => console.log(resposta))
```

Estrutura Básica do fetch()

A forma mais simples de usar o fetch:

```
fetch('https://api.exemplo.com/dados') .then(res => res.json()) .then(dados => console.log(dados)) .catch(erro => console.error(erro))
```



Primeiro .then() transforma a resposta em JSON.



Segundo .then() processa os dados.



.catch() captura erros da requisição.



Métodos HTTP com Fetch

Além do padrão GET, podemos enviar dados usando outros métodos HTTP:

```
fetch(url, { method: 'POST', body: JSON.stringify(dados)})
```

Principais métodos: **GET, POST, PUT, DELETE**. O corpo da requisição (body) deve estar no formato **JSON**.

Cabeçalhos (Headers) na Fetch API

Podemos enviar **informações extras** ao servidor usando headers:

```
fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(dados)})
```

Headers comuns: Content-Type, Authorization, Accept.





Tratamento de Erros

Fetch **não lança erro para respostas HTTP 4xx ou 5xx**. É necessário verificar o status manualmente:

```
fetch(url) .then(res => { if (!res.ok) throw new Error('Erro na requisição') return res.json() })
```


Uso de async/await

O async/await torna o código **mais sequencial e legível**:

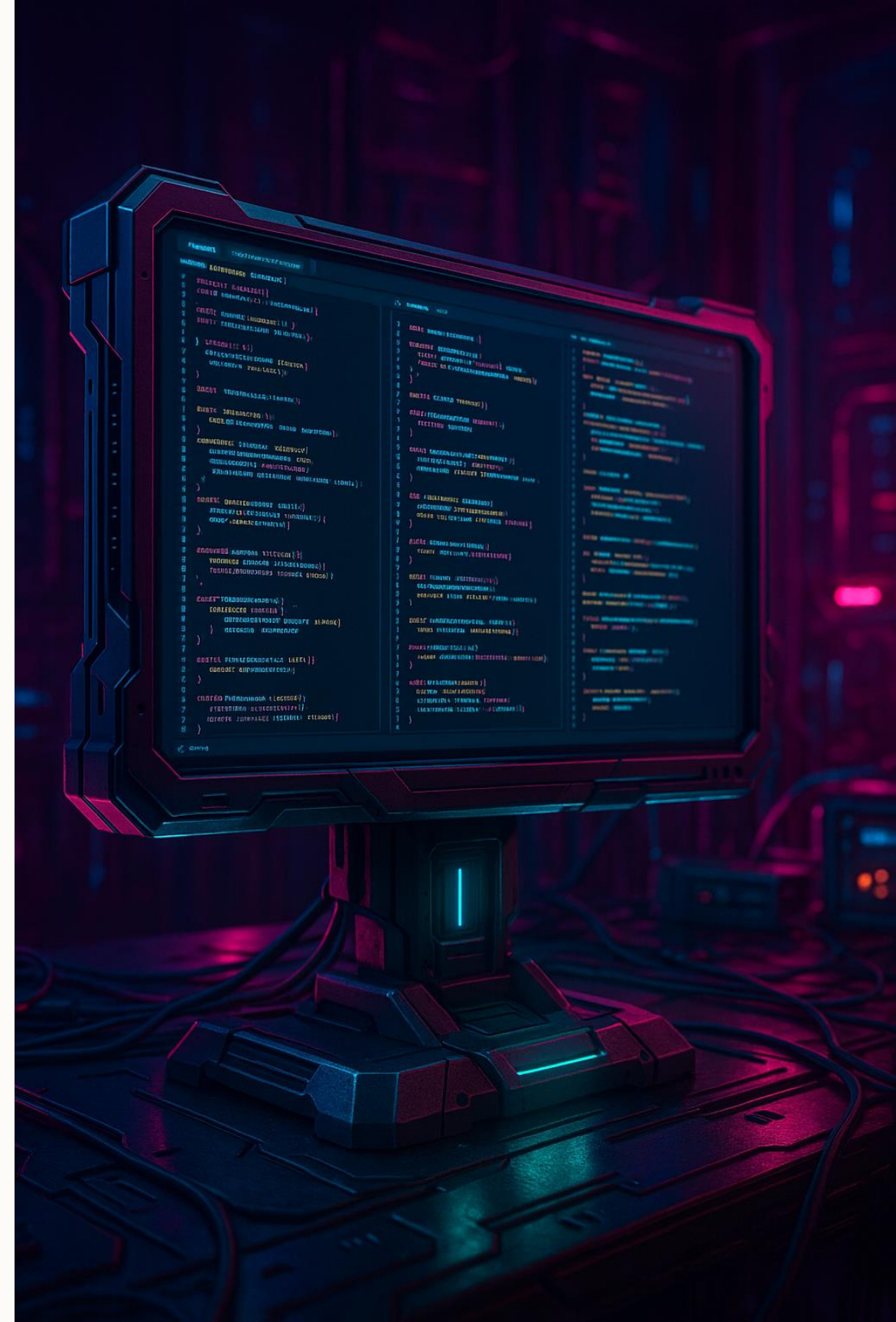
```
async function carregarDados() { try { let res = await fetch(url) let dados = await res.json() console.log(dados) } catch (erro) { console.error(erro) }}
```



Envio de Dados com POST

Para enviar informações ao servidor:

```
async function enviarDados() { let res = await fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ nome: 'Ana', idade: 22 }) }) let resposta = await res.json() console.log(resposta)}
```



Trabalhando com APIs Públicas

A Fetch API é perfeita para consumir **APIs externas**, por exemplo:

```
fetch('https://api.exemplo.com/usuarios') .then(res => res.json())  
  .then(dados => console.log(dados))
```

Não é necessário backend próprio, ideal para **aprendizado e testes rápidos**.



Boas Práticas com Fetch

Sempre trate erros com `.catch()` ou `try/catch`.

Transforme a resposta em JSON antes de usar.

Use `async/await` para código mais legível.

Configure headers corretamente para enviar e receber dados.

Evite duplicar chamadas desnecessárias para não sobrecarregar a API.

